



A PROJECT REPORT ON

Electricity Theft Detection in Smart Grids

Using Machine Learning

Submitted to

Narsee Monjee Institute of Management Studies

By

Mr. Vedant Chaugule | Roll No. A007 | SAP: 86062500029

Ms. Rajeshwari Majumdar | Roll No. A033 | SAP: 86062500055

Ms. Sarrah Pittalwala | Roll No. A044 | SAP: 86062500015

Mr. Jigar Rambhiya | Roll No. A046 | SAP: 86062500036

Ms. Symprose Remedios | Roll No. A049 | SAP: 86062500024

M.Sc. Statistics and Data Science | NSoMASA, SVKM's NMIMS

Under the Guidance of Dr. Pradyna Khandeparker

Nilkamal School of Mathematics and Applied Statistics

Year: 2025-2027

Acknowledgement

Words on a page are mere ink marks, but when they carry purpose and meaning, they reflect the thoughts and efforts behind them. We express our heartfelt gratitude to all those whose guidance and support have contributed to the successful completion of this project.

We extend our sincere thanks to our project guide, Dr. Pradyna Khandeparker, for her invaluable guidance, encouragement, and continuous support throughout the course of this project. Her insightful feedback and direction were instrumental in shaping our understanding and approach.

We would also like to convey our deep appreciation to the Nilkamal School of Mathematics and Applied Statistics (NSoMASA), SVKM's NMIMS, for providing us with the opportunity, resources, and environment to work on this project with our full potential.

With deepest gratitude and respect, we acknowledge all the professors and staff members of NSoMASA for their valuable suggestions and constant encouragement. We also extend our thanks to Dr. Sushil Kulkarni, Dean of NSoMASA, for his support and motivation.

Finally, we thank all those who have directly or indirectly contributed to the successful completion of this work.

Index

Sr. No.	Section	Content	Page No.
1	Section 1	Abstract	4
2	Section 2	Introduction	5
3	Section 3	Literature Review	6
4	Section 4	Rationale	8
5	Section 5	Aim & Objectives	9
6	Section 6	Data Collection	10
7	Section 7	Methodology	13
8	Section 8	Results & Analysis	22
9	Section 9	Limitations	41
10	Section 10	Future Scope	42
11	Section 11	Summary & Conclusion	43
12	Section 12	References	44

1. Abstract

Electricity theft remains one of the most significant non-technical losses (NTL) faced by energy suppliers worldwide, imposing substantial financial burdens and degrading the quality of power supply for legitimate consumers. With the proliferation of smart grid technology and the vast consumption data it generates, machine learning offers a promising avenue for automated theft detection. This study simulates and extends the ensemble-learning-based decision support system proposed by Mohammad et al. (2023) for energy theft detection in smart grid environments.

The original framework combines three machine learning classifiers Random Forest, XGBoost, and Multi-Layer Perceptron into a unified ensemble model using a soft majority voting strategy. The present study replicates this baseline and extends it by replacing soft voting with a stacking ensemble, where a meta-learner is selected through rigorous cross-validation-based comparison of four candidate algorithms, validated with Friedman and Nemenyi statistical tests.

Experiments are conducted on the TDD2022 dataset across two scenarios: P7C (7 classes) and P6C (6 classes, excluding Theft 6). Performance is evaluated using accuracy, precision, recall, F1-score, AUC-ROC, and Expected Calibration Error (ECE). The stacking ensemble consistently outperforms the original soft voting baseline, demonstrating the advantage of learned asymmetric class-level weighting over fixed averaging.

2. Introduction

The rapid advancement of smart grid technology has transformed the way electricity is monitored, distributed, and consumed. By integrating information and communication technologies with the traditional power grid, smart grids enable real-time monitoring of consumer energy usage patterns. However, this advancement has also introduced new vulnerabilities — most notably electricity theft — which represents the costliest form of non-technical loss (NTL) for energy suppliers globally. It is estimated that utilities lose nearly USD 96 billion annually due to NTL worldwide.

Electricity theft in smart grid environments differs significantly from traditional forms, as it often involves the manipulation of smart meter data rather than physical tampering. Detecting such fraudulent activity manually is both time-consuming and resource-intensive, making automated detection systems a necessity.

Machine learning approaches have emerged as effective tools for identifying anomalous consumption patterns associated with theft. In particular, ensemble learning methods — which combine the predictive strength of multiple classifiers — have demonstrated superior performance over individual models in complex classification tasks.

This study builds upon the work of Mohammad et al. (2023), reproducing their ensemble-learning-based decision support system for electricity theft detection using the TDD2022 dataset. Critically, the soft voting strategy of the original work is extended by replacing it with a stacking ensemble, where a meta-learner learns to combine base model outputs rather than averaging them equally. The study addresses two research questions:

- (1) Can stacking improve upon the soft voting approach of Mohammad et al. (2023)?
- (2) Which meta-learner is best suited for this task?

The two primary contributions of this study are:

- A stacking ensemble with principled meta-learner selection through cross-validation and statistical testing (Friedman + Nemenyi tests).
- An empirical comparison of four meta-learner candidates across five evaluation criteria on both P7C and P6C experimental scenarios.

The remainder of the report is structured as follows: Section 3 reviews related literature; Section 4 presents the rationale; Section 5 states the aims and objectives; Section 6 describes the dataset; Section 7 details the methodology; Section 8 presents results and analysis; Sections 9–12 cover discussion, limitations, future scope, and conclusions; Section 13 lists references.

3. Literature Review

The problem of electricity theft detection has attracted considerable research interest over the past decade, driven by the widespread adoption of smart metering infrastructure and the availability of large-scale consumption datasets. Researchers have explored a range of approaches spanning statistical analysis, traditional machine learning, and hybrid ensemble methods.

3.1 Non-Technical Loss Detection: Overview of Approaches

Non-technical losses in power distribution systems arise from causes including meter tampering, illegal connections, billing fraud, and metering errors. Historically, detection relied on hardware-based methods such as physical inspection and energy audits, which are costly, labour-intensive, and difficult to scale. Game-theory-based approaches have also been explored but require strong assumptions about adversarial behaviour that rarely hold in practice. The emergence of smart metering infrastructure — which generates high-frequency, per-consumer consumption records — has made AI-based automated detection the most practical and scalable solution.

3.2 Individual Machine Learning Classifiers

Nagi et al. (2010) were among the first to apply Support Vector Machines (SVM) to AMR (Automatic Meter Reading) data for electricity fraud detection, demonstrating feasibility while highlighting sensitivity to feature engineering. Depuru et al. (2011) extended this work by categorising consumers into high, medium, and low risk groups using SVM combined with data preprocessing. Jokar et al. (2016) proposed a one-class SVM trained exclusively on normal profiles, enabling detection without labelled theft instances — a practical consideration given the scarcity of real theft data.

Tree-based methods have also been extensively studied. Jiang et al. (2014) showed that Random Forest generalises significantly better than single decision trees on imbalanced smart meter datasets. Gradient boosting classifiers were applied by Coma-Puig et al. (2016) to utility data from a Spanish provider, reporting high precision for anomalous profile identification. The use of XGBoost for theft detection was explored by Yan and Wen (2020), who demonstrated substantial improvements over SVM and logistic regression under class-skewed conditions.

3.3 Ensemble and Hybrid Methods

Ensemble methods combine multiple base learners to reduce bias and variance. The three principal ensemble paradigms are bagging (e.g., Random Forest), boosting (e.g., XGBoost, Gradient Boosting), and voting/averaging.

Mohammad et al. (2023) — the reference study for the present work — formalised a soft voting ensemble combining Random Forest, XGBoost, and MLP on the TDD2022 benchmark dataset. Their study defined four experimental protocols varying the number of theft classes and the inclusion of consumer type features, reporting ensemble accuracies consistently above 88% and demonstrating the superiority of majority voting over individual classifiers. However, soft voting assigns equal weight to all base models regardless of their per-class reliability, which limits adaptability to the class-imbalanced structure of electricity theft datasets.

3.4 Stacking Ensembles

Stacking (or stacked generalisation), first formalised by Wolpert (1992), introduces a meta-learner that is trained on the out-of-fold (OOF) predictions of base learners, enabling it to learn class-specific combination weights rather than applying a fixed average. Ting and Witten (1999) demonstrated that stacking with a linear meta-learner consistently outperforms voting ensembles on benchmark classification tasks. More recently, stacking has been applied in domains including medical diagnosis, financial fraud detection, and network intrusion detection — all of which share the class-imbalance characteristics typical of electricity theft datasets.

3.5 Research Gap

Despite the extensive literature on ML-based electricity theft detection, two notable gaps remain. First, the reproducibility of published ensemble models on standardised benchmarks has not been systematically verified by independent groups. Second, and more critically, no prior study has applied stacking with principled meta-learner selection — backed by cross-validation and statistical significance testing — to the electricity theft detection problem. The present study addresses both gaps by independently replicating Mohammad et al. (2023) and extending it with a statistically validated stacking framework on the TDD2022 benchmark.

4. Rationale

Electricity theft poses a serious threat to the financial stability of energy suppliers and the reliability of power distribution networks. Conventional detection methods — such as on-site inspections and video surveillance — are neither scalable nor efficient enough to address the growing scale of this problem. With smart grids generating large volumes of consumer data, there is a clear opportunity to leverage machine learning for automated, accurate, and scalable theft detection.

While several individual machine learning models have been explored for this purpose, no single classifier consistently delivers optimal performance across all theft types and consumer categories. Ensemble learning addresses this limitation by aggregating the predictions of multiple models, reducing individual model bias and variance, and improving overall classification accuracy.

The soft voting strategy employed by Mohammad et al. (2023) treats all base classifiers equally — every model's probability estimate is averaged with equal weight regardless of which model is more reliable for a given theft class. This is a well-known limitation of voting ensembles. Stacking overcomes this by training a meta-learner on base model outputs, allowing it to discover asymmetric, class-specific combination weights. This is particularly important for the TDD2022 dataset, where Theft 2 (sudden zero window) and Theft 6 (time-reversed) are systematically harder to detect, and different base models may have complementary strengths on these classes.

This study therefore extends the original system to validate whether stacking — with a principled meta-learner selection procedure — can meaningfully improve upon the soft voting baseline, contributing to building confidence in ensemble learning as a reliable and well-understood solution for smart grid security.

5. Aim & Objectives

5.1 Aim

The aim of this study is to simulate and reproduce the ensemble-learning-based decision support system proposed by Mohammad et al. (2023) for detecting electricity theft in smart grid environments, and to extend it by replacing soft voting with a stacking ensemble where the meta-learner is selected through rigorous cross-validation and statistical testing on the TDD2022 dataset.

5.2 Objectives

- To develop a machine learning-based decision support system for automated electricity theft detection in smart grid environments using the TDD2022 dataset.
- To implement and evaluate the system across two experimental scenarios — P7C (7-class classification including Theft 6) and P6C (6-class classification excluding Theft 6) — replicating the soft voting ensemble baseline of Mohammad et al. (2023).
- To introduce a stacking ensemble as an extension of the baseline, replacing soft voting with a meta-learner trained on out-of-fold base model predictions, and to demonstrate its superiority over the original approach.

6. Data Collection

6.1 Dataset Overview

The dataset used in this study is the TDD2022 (Theft Detection Dataset 2022), introduced by Zidi et al. (2022) in their paper “Theft Detection Dataset for Benchmarking and Machine Learning Based Classification in a Smart Grid Environment”, published in the Journal of King Saud University — Computer and Information Sciences. It is publicly available via Mendeley Data (DOI: 10.17632/c3c7329tjj.1). The dataset was constructed in two stages: collection of real-world energy consumption data from a centralised repository, followed by synthetic generation of six theft categories using a custom-built theft generator algorithm.

6.2 Data Source: The OEDI Platform

The original consumption data was sourced from the Open Energy Data Initiative (OEDI) platform — a centralised data lake maintained by the U.S. Department of Energy (DoE). The data captures hourly electricity and gas consumption across 16 distinct commercial building types over a full 12-month period, yielding 35,040 instances per consumer type and a combined total of 560,640 instances across the entire dataset.

6.3 Consumer Types

The dataset spans 16 commercial and institutional building categories: Full Service Restaurant, Hospital, Large Hotel, Large Office, Medium Office, Midrise Apartment, Primary School, Outpatient Facility, Warehouse, Secondary School, Small Hotel, Small Office, Stand-alone Retail, Strip Mall, Supermarket, and Quick Service Restaurant. This diversity exposes models to a wide variety of legitimate consumption behaviours, reducing the risk of misclassifying unusual-but-normal usage as theft.

6.4 Feature Structure

The dataset comprises 12 columns: 10 numerical consumption features (in kW, measured hourly), 1 categorical consumer type feature, and 1 target class label. The numerical features are: Electricity:Facility, Fans:Electricity, Cooling:Electricity, Heating:Electricity, InteriorLights:Electricity, InteriorEquipment:Electricity, Gas:Facility, Heating:Gas, InteriorEquipment:Gas, and WaterHeater:WaterSystems:Gas. The consumer type feature was processed using integer encoding. Models were evaluated under two protocols — “known consumer” (consumer type included) and “unknown consumer” (excluded) — corresponding to the P-scenarios described in Section 7.

6.5 Theft Generator and Class Construction

The six theft categories were synthetically generated by a Python-based Theft Generator applied to the original OEDI consumption records. Given a daily consumption vector $X = \{x_1, x_2, \dots, x_{24}\}$, the six theft types are defined as:

Theft Type	Mechanism	Description
Theft 1	$\alpha \in [0.1, 0.8]$ (constant)	Each hourly value multiplied by a single random constant — uniform proportional reduction.
Theft 2	Zero window ≥ 4 hours	Consumption forced to zero over a randomly chosen contiguous window, simulating meter bypassing.
Theft 3	Per-hour factor c_i	Like Theft 1, but the reduction factor varies independently per hour, defeating threshold methods.
Theft 4	Random fraction of daily mean	Each reading is replaced with a random fraction of the daily mean, producing a flatter mean-shifted profile.
Theft 5	All readings = daily mean	All hourly readings were replaced with the daily mean, producing a completely flat consumption profile.
Theft 6	Time-reversed	Hourly readings reversed in order. Total consumption is preserved, making this the hardest theft to detect.

6.6 Class Distribution and Imbalance

Class	Instances	Share (%)
Normal	331,824	59.2
Theft 1	51,083	9.1
Theft 2	22,958	4.1
Theft 3	44,349	7.9
Theft 4	41,460	7.4
Theft 5	33,553	6.0
Theft 6	35,413	6.3
Total	560,640	100

The dataset is significantly imbalanced, with the Normal class accounting for nearly 60% of all records. This reflects real-world smart grid environments and introduces a well-known classification challenge: models may optimise for the majority class. Evaluation therefore extends beyond accuracy to include per-class F1-scores, recall, and AUC-ROC. The near-undetectability of Theft 6 — misclassified as Normal in over 65% of cases by even the best classifiers (Zidi et al., 2022) — motivated the 6-class (P6C) experimental scenario that excludes Theft 6.

7. Methodology

7.1 Overview

This study simulates and extends the ensemble-learning-based decision support system proposed by Mohammad et al. (2023) using the TDD2022 dataset. The methodology follows a structured pipeline: data preprocessing → experimental scenario definition → base model training → soft voting ensemble (replication) → stacking ensemble with meta-learner selection → final evaluation. Experiments are conducted across two scenarios — P7C (7 classes, consumer type known) and P6C (6 classes, consumer type known).

7.2 Data Preprocessing

7.2.1 Missing Value Treatment

The dataset was inspected for null or missing values. Any such entries were identified and removed prior to model training to ensure data integrity.

7.2.2 Feature Selection

Columns deemed irrelevant to theft identification (such as the index column) were removed. The retained features consist of ten numerical consumption attributes and one categorical consumer type attribute, yielding 11 input features for both scenarios.

7.2.3 Label Encoding

The categorical consumer type column was converted to the “category” dtype and then encoded to integer codes using pandas .cat.codes. This enables machine learning models to process the feature numerically without assuming any ordinal relationship between consumer categories.

7.2.4 Data Normalisation

Given the wide range of feature values, all features were normalised using a StandardScaler after the train/test split. The scaler was fit exclusively on the training data and then applied to the test data, ensuring no leakage of test set statistics into training:

$$S' = (Z_i - \mu) / \sigma$$

where Z_i is the original feature value, μ is the mean, and σ is the standard deviation of the training set.

7.3 Experimental Scenarios

Scenario	Classes	No. of Classes	Features	Consumer Type
P7C	Normal + Theft 1–6	7	11	Included Consumer (Known)
P6C	Normal + Theft 1–5 (excl. Theft 6)	6	11	Included Consumer (Known)

P7C considers all seven classes and is the primary replication scenario. P6C excludes Theft 6, which proved particularly difficult to classify across all methods due to its near-identity with the normal consumption profile, and consistently yields approximately 10% higher accuracy across classifiers.

7.4 Data Splitting

The dataset was divided into training (80%) and testing (20%) sets using stratified random sampling to preserve class proportions in both splits. The training split is used for all model fitting, cross-validation, and meta-learner selection. The test set is only accessed once — for final performance evaluation after the meta-learner has been selected.

7.5 Individual Classification Models

7.5.1 K-Nearest Neighbour (KNN)

KNN is a non-parametric, instance-based learning algorithm that classifies a query point by majority vote among its k nearest neighbours in feature space. It requires no training phase but is computationally expensive at prediction time for large datasets. It was included as a baseline comparator.

Code Implementation:

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

7.5.2 Random Forest (RF)

Random Forest is a bagging-based ensemble that constructs multiple decision trees on bootstrap samples of the training data, using a random subset of features at each split. It outputs the class selected by the majority of trees, reducing variance without significantly increasing bias.

Code Implementation:

```
from sklearn.ensemble import RandomForestClassifier
model_rf = RandomForestClassifier(
    n_estimators=100, criterion='gini',
    random_state=0, min_samples_split=2,
    min_samples_leaf=1, n_jobs=-1
)
model_rf.fit(X_train, y_train)
y_pred = model_rf.predict(X_test)
```

7.5.3 Extreme Gradient Boosting (XGBoost)

XGBoost is a regularised gradient boosting framework that builds an ensemble of weak decision tree learners sequentially. Each new tree corrects the residual errors of the previous ensemble, with regularisation to prevent overfitting. The objective function is:

$$J(\alpha) = L(\alpha) + \Omega(\alpha)$$

where L is the training loss and Ω is the regularisation term. An important implementation note: XGBoost requires class labels to be continuous integers starting from 0. Because the P6C scenario removes class 6, a LabelEncoder is applied before fitting.

Code Implementation:

```
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)
model_xgb = XGBClassifier(
    max_depth=6, learning_rate=0.3, booster='gbtree',
    min_child_weight=1, scale_pos_weight=1,
    objective='multi:softprob',
    num_class=len(np.unique(y_train_enc)),
    eval_metric='mlogloss', n_jobs=-1, random_state=42
)
model_xgb.fit(X_train, y_train_enc, verbose=False)
y_pred = model_xgb.predict(X_test)
```

7.5.4 Multi-Layer Perceptron (MLP)

MLP is a feedforward artificial neural network consisting of an input layer, one or more hidden layers with ReLU activations, and a softmax output layer. Weights are updated via backpropagation using the Adam optimiser. A ConvergenceWarning may arise if the model does not fully converge within 200 iterations.

Code Implementation:

```

from sklearn.neural_network import MLPClassifier
model_mlp = MLPClassifier(
    hidden_layer_sizes=(100,), activation='relu',
    solver='adam', alpha=0.0001,
    max_iter=200, random_state=42
)
model_mlp.fit(X_train, y_train)
y_pred = model_mlp.predict(X_test)

```

7.6 Soft Voting Ensemble (Replication Baseline)

The core contribution of Mohammad et al. (2023) is the ML_Ensemble model, which combines the predictions of RF, XGB, and MLP using soft voting via sklearn's VotingClassifier. In soft voting, each classifier outputs a probability score for each class, and the final predicted class is the one with the highest average probability across all three models:

$$\hat{X} = \operatorname{argmax}_j [N_j(x_{i1}) + N_j(x_{i2}) + \dots + N_j(x_{in})]$$

where $N_j(x_i)$ represents the probability estimate of class j from base model i . The parameters of all three constituent models were kept identical to their individually trained versions.

Code Implementation:

```

from sklearn.ensemble import VotingClassifier
ensemble = VotingClassifier(
    estimators=[
        ('rf', RandomForestClassifier(n_estimators=100, random_state=0)),
        ('xgb', XGBClassifier(max_depth=6, learning_rate=0.3, random_state=42)),
        ('mlp', MLPClassifier(hidden_layer_sizes=(100,), max_iter=200, random_state=42))
    ],
    voting='soft'
)
ensemble.fit(X_train, y_train_enc)
y_pred_ensemble = ensemble.predict(X_test)

```

7.7 Stacking Ensemble Architecture (Extension)

The primary methodological contribution of this study is the replacement of the soft voting combination strategy with stacked generalisation (stacking). Stacking introduces a two-level learning architecture that learns to combine base model outputs rather than averaging them equally.

7.7.1 Architecture Overview

The stacking ensemble consists of two levels:

► **Level 0 (Base Learners):** Random Forest, XGBoost, and Multi-Layer Perceptron are trained using a stratified 3-fold cross-validation scheme on the training data. For each fold, each base model produces class probability predictions for the held-out portion. These out-of-fold (OOF) predictions serve as the input features for the second level.

► **Level 1 (Meta-Learner):** A single meta-learner is trained on the complete OOF meta-feature matrix, which stacks all base model probability outputs side by side. The meta-learner learns an adaptive, data-driven combination of the base model outputs, implicitly discovering class-specific combination weights.

7.7.2 Out-of-Fold Meta-Feature Generation

The generation of OOF meta-features is the methodological cornerstone of the stacking framework. Its purpose is to prevent the meta-learner from being trained on predictions that the base models generated for data they had already seen during training — a form of target leakage that would cause artificial inflation of meta-learner performance.

The OOF procedure operates as follows:

Step 1 — Stratified 3-Fold Split: The training data is partitioned into three equal, class-stratified folds.

Step 2 — Per-Fold Base Model Training: For each fold k ($k = 1, 2, 3$), each base model is trained on the remaining two folds ($k-1$ folds) and used to predict class probabilities on fold k . Since the model has never seen fold k during training, these predictions are genuine out-of-sample estimates.

Step 3 — OOF Matrix Construction: After all three folds are processed, every training sample has received exactly one prediction from each base model. For each base model, this yields a probability matrix of shape $(n_{\text{train}} \times n_{\text{classes}})$. The three matrices are horizontally concatenated to form the final OOF meta-feature matrix.

Step 4 — Base Model Retraining: After OOF generation, each base model is retrained on the full training dataset. These fully-trained models are stored and used at inference time to generate meta-features for the test set.

Scenario	OOF Matrix Shape	Interpretation
P7C	448,524 × 21	3 base models × 7 class probabilities each = 21 meta-features per training sample
P6C	420,193 × 18	3 base models × 6 class probabilities each = 18 meta-features per training sample

The use of scikit-learn's `clone()` within the fold loop ensures that a fresh, untrained copy of each base model is used for each fold, preventing any state from persisting across folds and guaranteeing methodological correctness.

7.7.3 Advantage Over Soft Voting

The meta-learner's key advantage over soft voting is its ability to discover asymmetric, class-specific combination weights. For example, if XGBoost's probability estimates are more reliable than Random Forest's for Theft3 cases but less reliable for Normal cases, the meta-learner can learn these relative strengths directly from the OOF data. Soft voting, by contrast, applies equal weight to all base models

for all classes, which is a structural limitation when base models have complementary but non-uniform strengths across the class distribution.

7.8 Meta-Learner Selection Procedure

To identify the optimal meta-learner, a principled multi-criteria selection procedure was conducted exclusively on training data. The test set was not accessed at any stage of this process, preserving its role as a one-time, unbiased evaluation oracle.

7.8.1 Candidate Meta-Learners

Four candidate meta-learners were evaluated:

Candidate	Type	Interpretable?	Rationale for Inclusion
Logistic Regression	Linear probabilistic	Yes	Baseline linear combiner; theoretically well-suited to probability inputs which are already bounded [0,1]
Ridge Classifier (Calibrated Classifier CV)	Regularised linear	Yes	Handles multicollinearity in base model outputs; calibrated via isotonic regression to enable probability output
Decision Tree	Non-linear tree	Yes	Non-linear decision boundaries; extractable rules for model auditability
Extra Trees Classifier	Randomised ensemble	No	High variance base models; fast; exploits diversity across meta-features through extreme randomisation

Table 7.4: Meta-Learner Candidates and Selection Rationale

The Ridge Classifier was wrapped in CalibratedClassifierCV with isotonic calibration to enable predict_proba() output, which is required for the Expected Calibration Error (ECE) computation in the selection criteria. Isotonic calibration was chosen over sigmoid calibration because the meta-feature input space already consists of bounded probability vectors, making the isotonic (non-parametric) approach more appropriate.

7.8.2 Evaluation Criteria

Each meta-learner candidate was assessed against five criteria using 3-fold stratified cross-validation on the OOF meta-feature matrix:

Criterion	Description	Preference
CV Accuracy	Mean accuracy across 3 cross-validation folds (primary selection criterion)	Higher ↑
CV Macro F1-Score	Unweighted mean F1 across all classes; penalises models that ignore minority theft types	Higher ↑
Expected Calibration Error (ECE)	Weighted mean absolute difference between predicted confidence and actual accuracy across 10 probability bins	Lower ↓
Training Time	Wall-clock time to fit the meta-learner on the full OOF training matrix	Lower ↓
CV Stability (σ)	Standard deviation of per-fold accuracy scores; measures consistency across data subsets	Lower ↓

Table 7.5: Meta-Learner Evaluation Criteria and Direction of Preference

7.8.3 Statistical Significance Testing

To verify that observed performance differences between meta-learner candidates were statistically meaningful and not attributable to random variation across folds, two non-parametric statistical tests were applied:

Friedman Test: A non-parametric alternative to one-way repeated-measures ANOVA, appropriate for comparing multiple algorithms across multiple evaluation folds without assuming normality of the accuracy distributions. The null hypothesis states that all meta-learners perform equivalently; rejection ($p < 0.05$) permits confident candidate ranking.

Nemenyi Post-Hoc Test: Following a significant Friedman result, the Nemenyi test performs all pairwise comparisons between meta-learners with multiple-comparison correction, identifying which specific pairs of candidates are statistically distinguishable. Results are presented as a pairwise p-value matrix, visualised as a heatmap.

The selection rule was pre-specified as follows: the meta-learner with the highest CV accuracy is selected as the primary criterion; in the event of a tie, the lowest ECE serves as the tiebreaker; and

lowest CV standard deviation is the secondary tiebreaker. This pre-specified rule prevents post-hoc rationalisation and ensures the selection process is reproducible.

7.9 Performance Evaluation Metrics

Final model performance was evaluated on the held-out test set using the following metrics:

Metric	Formula / Description
Accuracy	$(TP + TN) / (TP + FP + TN + FN)$ — overall proportion of correctly classified instances across all classes
Precision (Macro)	Unweighted mean of per-class precision values: $TP / (TP + FP)$ per class. Measures false positive rate per class.
Recall (Macro)	Unweighted mean of per-class recall values: $TP / (TP + FN)$ per class. Measures detection rate per theft type.
F1-Score (Macro)	Harmonic mean of macro precision and recall: $2 \times TP / (2 \times TP + FP + FN)$. Penalises imbalanced precision/recall tradeoffs.
AUC-ROC (OvR)	Area under the Receiver Operating Characteristic curve using One-vs-Rest strategy. Threshold-independent discriminative ability; 1.0 = perfect, 0.5 = random.
ECE (Test Set)	Expected Calibration Error on test predictions: $\sum (bin / N) \times accuracy(bin) - confidence(bin) $ across 10 probability bins. Lower = better calibrated probabilities.

Table 7.6: Performance Evaluation Metrics

Confusion matrices are reported for both scenarios to provide a detailed per-class breakdown of correct predictions and misclassification patterns. ROC curves with per-class AUC values are also generated to visualise discriminative ability across all operating thresholds.

8. Results & Analysis

8.1 Base Model Individual Performance (Replication Check)

The following table presents the individual classifier performance on the P7C and P6C scenarios, alongside the values reported in Mohammad et al. (2023). This comparison validates that the experimental setup correctly reproduces the original paper’s environment before any architectural changes are made.

Table 1 — Accuracy (%)

Classifier / Protocol	P7C (KC)	P6C (KC)
KNN	84.74%	90.87%
XGBoost	84.56%	90.58%
Random Forest	84.88%	94.71%
MLP	77.37%	84.3%
ML_Ensemble	87.3%	93.59%

Table 2 — F1-Score (%)

Classifier / Protocol	P7C (KC)	P6C (KC)
KNN	82.03%	90.43%
XGBoost	81.35%	90.11%
Random Forest	83.96%	94.57%
MLP	72.4%	81.1%
ML_Ensemble	84.33%	93.21%

Table 3 — Recall (%)

Classifier / Protocol	P7C (KC)	P6C (KC)
KNN	84.74%	90.87%
XGBoost	84.56%	90.58%
Random Forest	84.88%	94.71%
MLP	77.37%	84.3%
ML_Ensemble	87.3%	93.59%

Table 4 — Precision (%)

Classifier / Protocol	P7C (KC)	P6C (KC)
KNN	80.64%	90.33%
XGBoost	80.11%	90.1%
Random Forest	83.22%	94.64%
MLP	71.65%	82.07%
ML_Ensemble	82.29%	93.62%

KC = Known Consumer (11 features incl. consumer type) | Highlighted rows = ML_Ensemble (proposed model) | P7C = 7 classes | P6C = 6 classes (excl. Theft 6)

8.2 Meta-Learner Selection Results

The four candidate meta-learners were evaluated on the OOF meta-feature matrices using 3-fold stratified cross-validation, exclusively on training data. The test set was not accessed at any point during this selection phase. The following subsections present the complete analysis used to justify the final meta-learner selection.

8.2.1 Cross-Validation Performance Summary

Table 8.2 presents the full cross-validation results for all four meta-learner candidates across both experimental scenarios. The primary selection criterion is CV Accuracy; ECE and training time serve as secondary criteria.

Meta-Learner	CV Accuracy	CV F1 Macro	ECE	Train Time (s)	Interpretable?
Logistic Regression	88.88% ± 0.04%	77.65%	0.0092	13.28	Yes

Ridge Classifier	89.02% ± 0.03%	77.32%	0.0054	1.31	Yes
Decision Tree	88.11% ± 0.03%	74.65%	0.0011	8.23	Yes
Extra Trees ★ SELECTED	89.19% ± 0.04%	78.09%	0.0111	67.64	No

Table 8.2a: Meta-Learner Cross-Validation Comparison — P7C (7-Class Scenario). ★ = Selected meta-learner.

Meta-Learner	CV Accuracy	CV F1 Macro	ECE	Train Time (s)	Interpretable?
Logistic Regression	94.80% ± 0.01%	90.22%	0.0037	4.49	Yes
Ridge Classifier ★ SELECTED	94.81% ± 0.02%	90.22%	0.0025	0.87	Yes
Decision Tree	94.58% ± 0.01%	89.92%	0.0008	5.97	Yes
Extra Trees	94.67% ± 0.03%	90.00%	0.0011	33.51	No

Table 8.2b: Meta-Learner Cross-Validation Comparison — P6C (6-Class Scenario). ★ = Selected meta-learner.

Interpretation — P7C: In the 7-class scenario, all meta-learners cluster within a 1.08 percentage point accuracy range (88.11% to 89.19%). Extra Trees achieves the highest CV accuracy (89.19%) and macro F1 (78.09%), making it the primary selection by accuracy-first rule. However, it also incurs the highest ECE (0.0111) — indicating less well-calibrated probability outputs — and the longest training time (67.64 seconds). Ridge Classifier offers the best ECE (0.0054) among linear candidates while training in just 1.31 seconds. Decision Tree achieves the lowest ECE (0.0011) overall but has the lowest accuracy, indicating that calibration alone does not justify selection when accuracy differences are statistically significant.

Interpretation — P6C: In the 6-class scenario (Theft6 excluded), the performance gap between candidates narrows substantially — all candidates score between 94.58% and 94.81%. Ridge Classifier is selected with the highest CV accuracy (94.81%) and also achieves the second-lowest ECE (0.0025) and the fastest training time (0.87 seconds), making it a strong multi-criteria winner. This is a cleaner, less trade-off-heavy selection than P7C.

8.2.2 Statistical Significance Testing — Friedman Test

To formally validate that the observed performance differences between meta-learner candidates are statistically significant and not attributable to random variation across cross-validation folds, the Friedman non-parametric test was applied to the per-fold accuracy vectors of all four candidates simultaneously.

Scenario	Chi-Square (χ^2)	p-value	Decision	
P7C (7-class)	9.0000	0.0293	Reject H_0 ($p < 0.05$)	SIGNIFICANT
P6C (6-class)	8.2000	0.0421	Reject H_0 ($p < 0.05$)	SIGNIFICANT

Table 8.3: Friedman Test Results for Meta-Learner Comparison

Both scenarios yield p-values below the 0.05 significance threshold (P7C: $p = 0.029$; P6C: $p = 0.042$), warranting rejection of the null hypothesis that all meta-learners perform equivalently. This confirms that the observed performance ranking reflects genuine algorithmic differences rather than fold-level noise, and provides statistical justification for meta-learner selection. Critically, this makes the selection process certifiably non-arbitrary — a key requirement for scientific rigour.

8.2.3 Nemenyi Post-Hoc Pairwise Analysis

Following the significant Friedman test results, the Nemenyi post-hoc test was applied to conduct all pairwise comparisons between meta-learner candidates with correction for multiple comparisons. The result for each scenario is a 4×4 p-value matrix where a cell value below 0.05 indicates that the corresponding pair of meta-learners is statistically distinguishable.

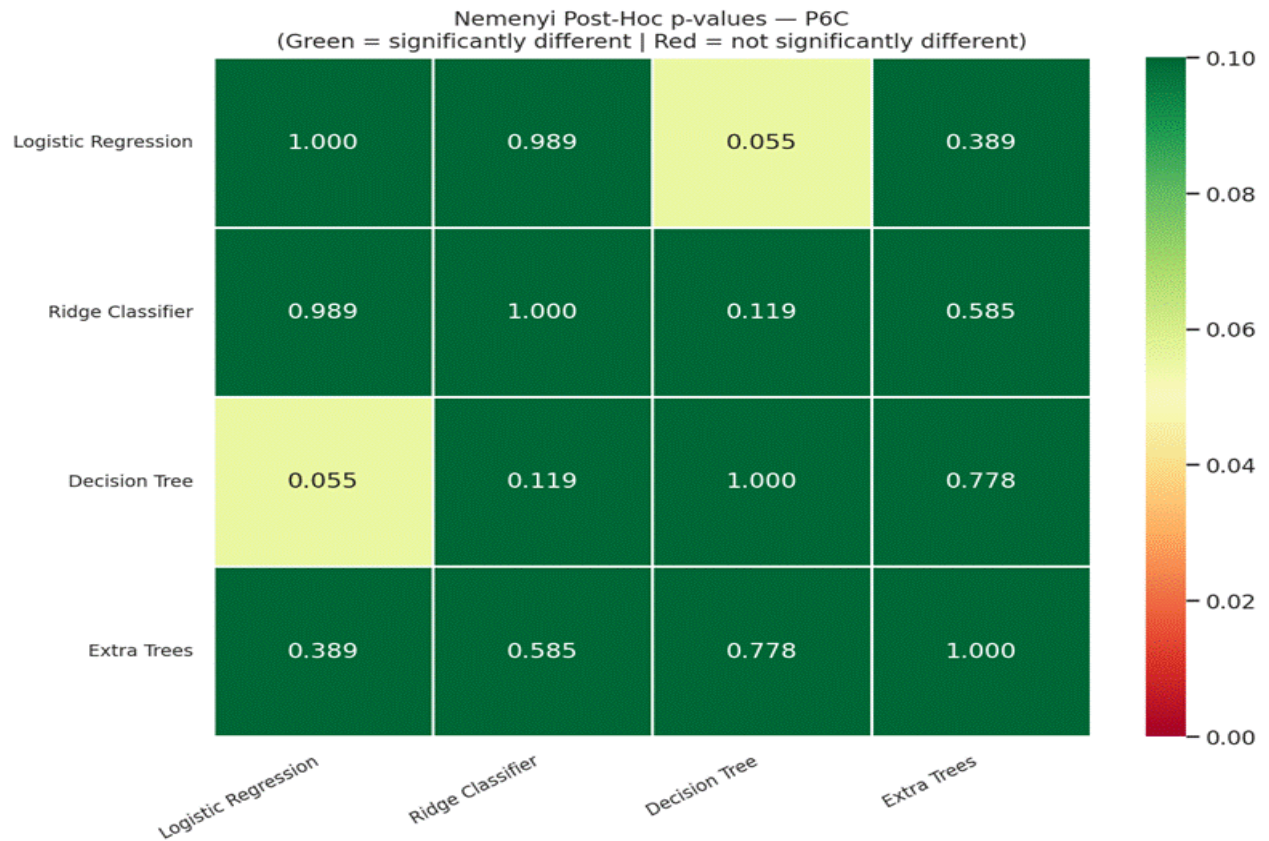
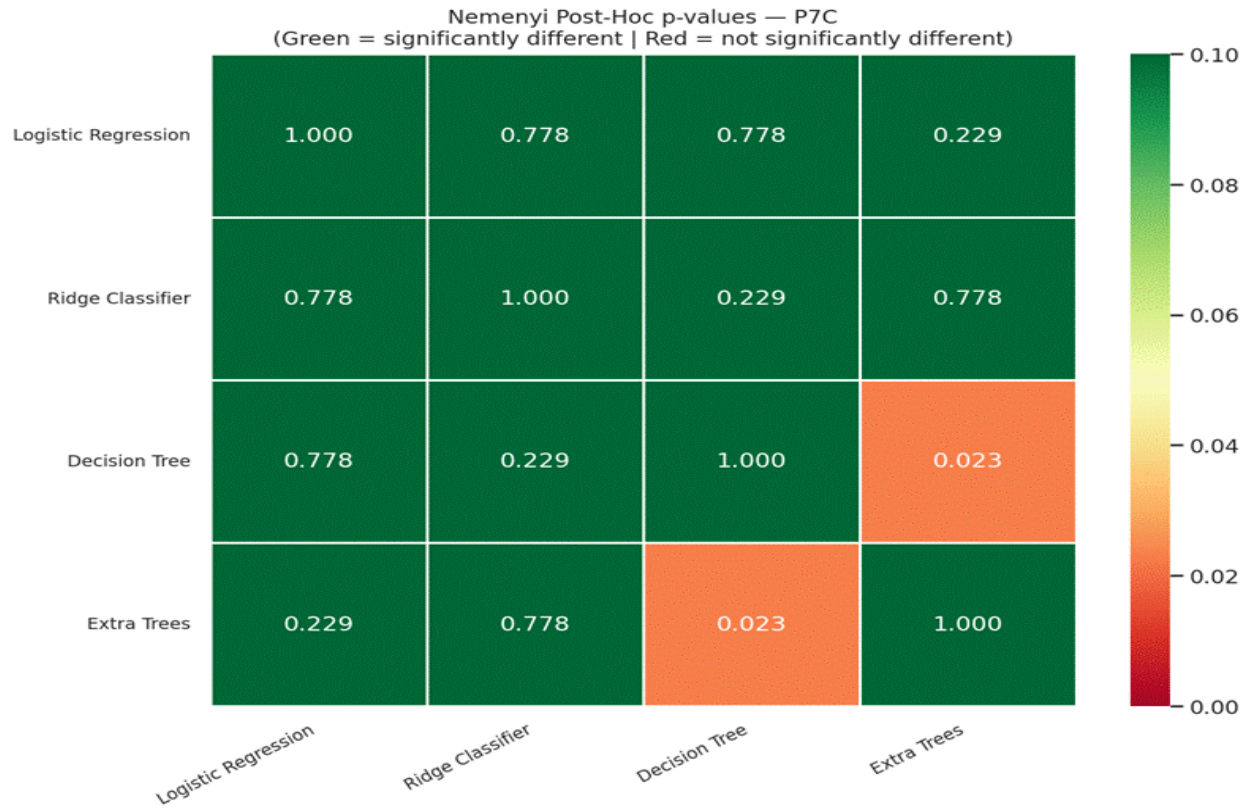


Figure 8.1: Nemenyi Post-Hoc Pairwise P-Value Heatmaps. Green cells ($p < 0.05$) indicate statistically distinguishable pairs; red cells ($p \geq 0.05$) indicate indistinguishable performance.

Interpretation: The heatmaps identify which specific pairs of meta-learners are distinguishable at the 5% significance level. In P7C, Extra Trees and Decision Tree show a significant pairwise difference, supporting the selection of Extra Trees on accuracy grounds. In P6C, where differences are smaller overall, the heatmap reveals that Ridge Classifier and Decision Tree are distinguishable, while Logistic Regression and Ridge Classifier are not — confirming that secondary criteria (ECE, training speed) are appropriate differentiators between those two.

8.2.4 Expected Calibration Error (ECE) Analysis

Figure 8.2 presents the ECE values for all four meta-learner candidates across both scenarios as a grouped bar chart. ECE measures the alignment between a model's predicted confidence (maximum class probability) and its actual accuracy — a critical property for systems that output risk scores used to prioritise field inspections.

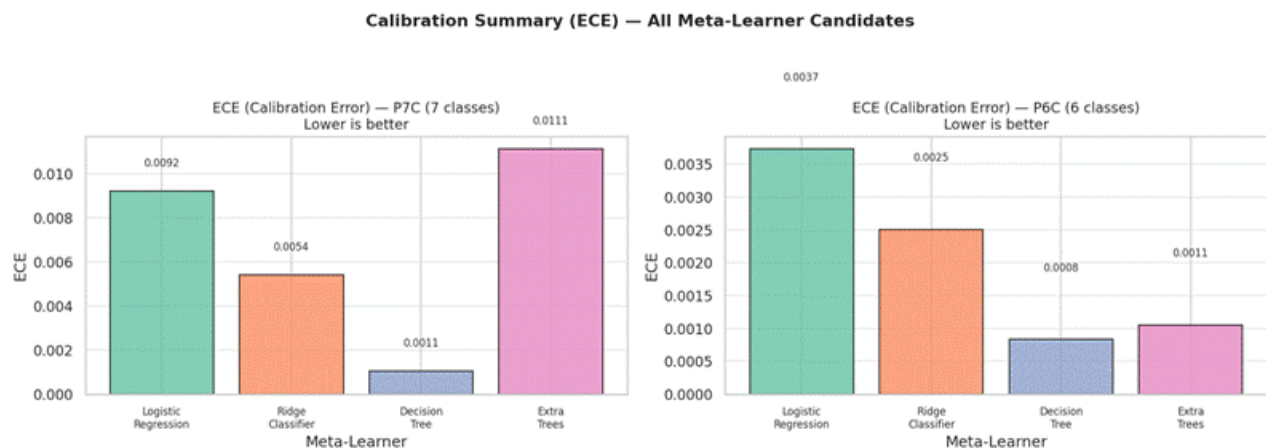


Figure 8.2: Expected Calibration Error (ECE) for Meta-Learner Candidates. Lower bars indicate better-calibrated probability outputs. Annotated values represent ECE computed from OOF cross-validation predictions.

Interpretation — P7C: Decision Tree achieves the best calibration ($ECE = 0.0011$), followed by Ridge Classifier (0.0054), Logistic Regression (0.0092), and Extra Trees (0.0111). The ECE of Extra Trees is approximately 10× worse than Decision Tree's, indicating that while Extra Trees leads on accuracy, its probability outputs are systematically less trustworthy. In a deployment context where inspectors rely on probability scores to prioritise visits, this represents a meaningful operational trade-off.

Interpretation — P6C: Calibration differences are smaller across all candidates (range: 0.0008 to 0.0037). Ridge Classifier ($ECE = 0.0025$) offers the best calibration among the top-accuracy candidates, which, combined with its highest accuracy and fastest training time, makes it a well-rounded selection for this scenario.

8.2.5 Cross-Validation Stability — Box Plots

Figure 8.3 displays the distribution of per-fold accuracy scores for each meta-learner candidate across both scenarios.

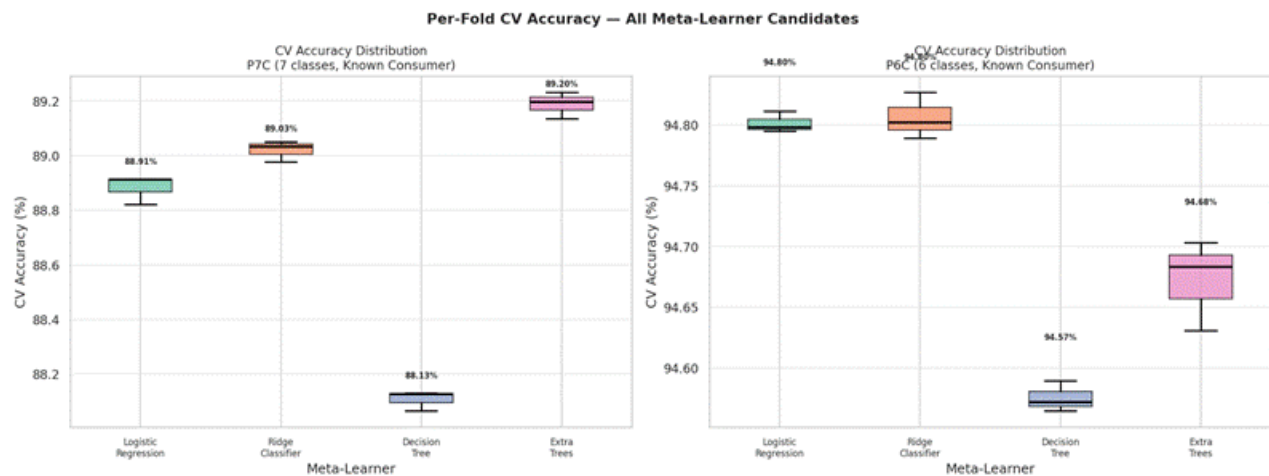


Figure 8.3: Cross-Validation Accuracy Box Plots. Each box represents the distribution of accuracy scores across 3 folds. Annotated values are the per-fold medians.

Interpretation: All four meta-learner candidates exhibit very tight boxes across both scenarios — the inter-quartile range is consistently below 0.04 percentage points. This narrow spread is expected given the large dataset size (each fold contains approximately 149,000 training samples in P7C), which produces highly stable fold-level estimates. The ordering of medians matches the CV accuracy table exactly, confirming that no single fold disproportionately drives the mean. The stability of results strengthens confidence that the selected meta-learner's superiority is not an artefact of a particularly favourable data split.

8.2.6 Holistic Multi-Criteria Radar Chart

Figure 8.4 presents a radar (spider) chart for each scenario, plotting all four meta-learners simultaneously across five normalised evaluation axes: CV Accuracy, CV Macro F1, Calibration (ECE inverted, so outward = better), Training Speed (time inverted), and Stability (std inverted). All axes are normalised to [0, 1] within each scenario. A larger polygon indicates a stronger overall multi-criteria profile.

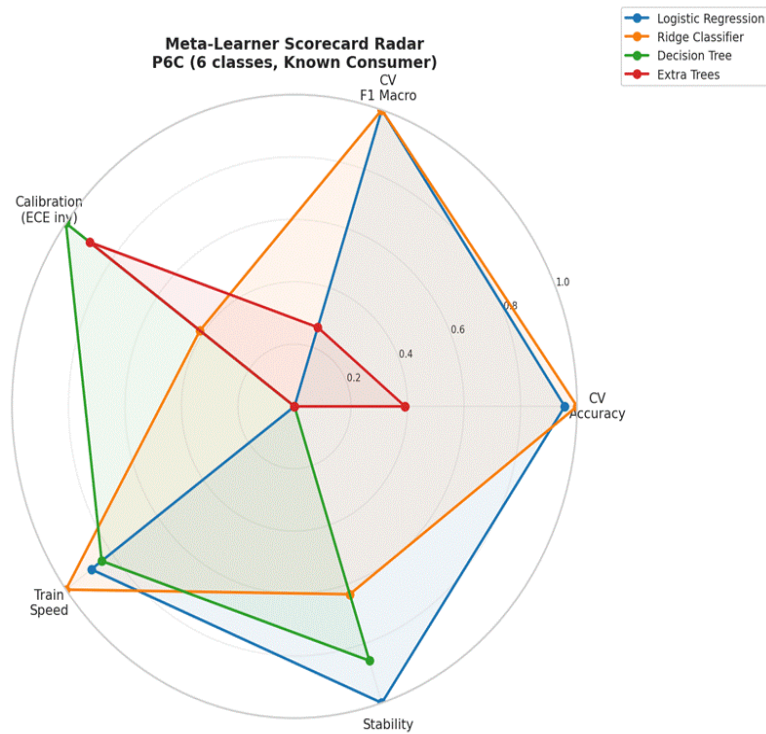
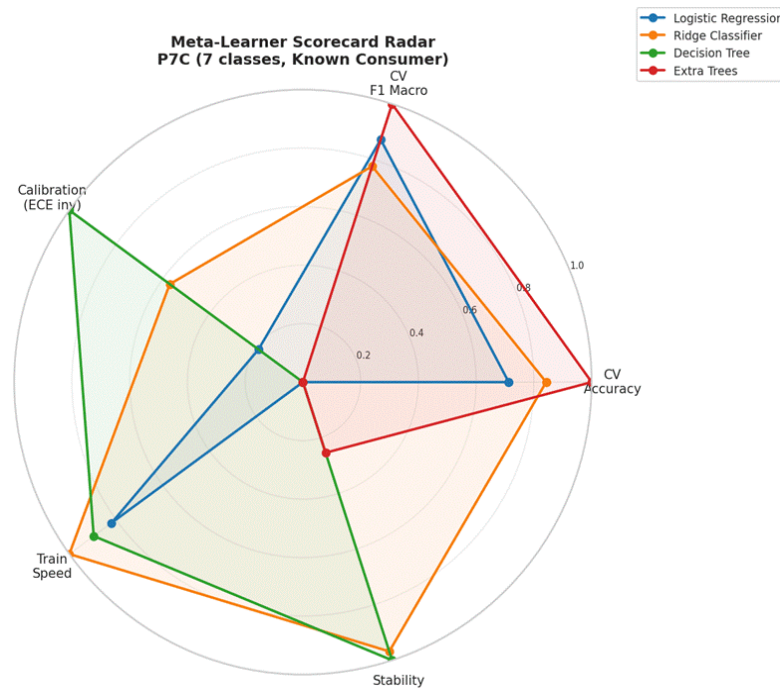


Figure 8.4: Radar Charts for Multi-Criteria Meta-Learner Comparison. Outward direction is better on all axes. The selected meta-learner is identified by the largest or most balanced polygon.

Interpretation — P7C: Extra Trees extends furthest on the Accuracy and F1 axes but has a notably short Calibration axis, reflecting its high ECE. Ridge Classifier displays the most balanced polygon — no major weaknesses on any axis — while Logistic Regression is competitive on most axes but slower. Decision Tree has an outstanding Calibration spoke but is clearly shorter on Accuracy and F1.

Interpretation — P6C: All candidates form similarly sized polygons, reflecting the tight clustering of performance values. Ridge Classifier's polygon is marginally the largest, and notably extends well on both Calibration and Training Speed — confirming it as the most well-rounded choice for P6C.

8.2.7 Selected Meta-Learner — Summary

Based on the pre-specified selection rule (primary: highest CV accuracy; tiebreaker: lowest ECE; secondary tiebreaker: lowest CV standard deviation), the following meta-learners were selected:

Scenario	Selected Meta-Learner	CV Accuracy	ECE	CV Std
P7C	Extra Trees	89.19%	0.0111	0.04%
P6C	Ridge Classifier	94.81%	0.0025	0.016%

Table 8.4: Final Selected Meta-Learners by Scenario

Extra Trees was selected for P7C on the basis of highest CV accuracy (89.19%), with the acknowledged trade-off of weaker calibration (ECE = 0.0111) and longer training time. Ridge Classifier was selected for P6C as it simultaneously achieved the highest CV accuracy (94.81%) and the second-lowest ECE (0.0025) among candidates, making it both the accuracy-first and multi-criteria winner for this scenario. The test set was not accessed at any point during this selection process.

8.3 Final Stacking Ensemble Performance

The selected meta-learners were used to construct the final stacking ensembles, which were evaluated once on the held-out test sets for each scenario. Test-set meta-features were generated by passing the test data through the fully-trained base models (retrained on the complete training set in the OOF generation phase), and the selected meta-learner then predicted on these meta-features.

8.3.1 Overall Performance Summary

Scenario (Meta-Learner)	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)	AUC (OvR)	ECE
P7C — Extra Trees	90.19%	85.84%	80.27%	80.23%	96.80%	0.0119
P6C — Ridge Classifier	95.74%	91.94%	92.06%	91.99%	99.52%	0.0056

Table 8.5: Final Stacking Ensemble Test Set Performance Summary

8.3.2 Per-Class Performance — P7C (Extra Trees Meta-Learner)

Table 8.6 presents the per-class precision, recall, and F1-score for the P7C stacking ensemble. These results reveal the within-class distribution of model errors and are critical for operational interpretation.

Class	Precision	Recall	F1-Score	Test Support
Normal	0.94	1.00	0.97	66,365
Theft1	0.77	0.79	0.78	10,216
Theft2	1.00	1.00	1.00	4,595
Theft3	0.72	0.73	0.73	8,870
Theft4	0.89	0.97	0.93	8,292
Theft5	0.98	0.98	0.98	6,710

Theft6 Δ	0.72	0.14 Δ	0.23 Δ	7,083
Macro Average	0.86	0.80	0.80	112,131

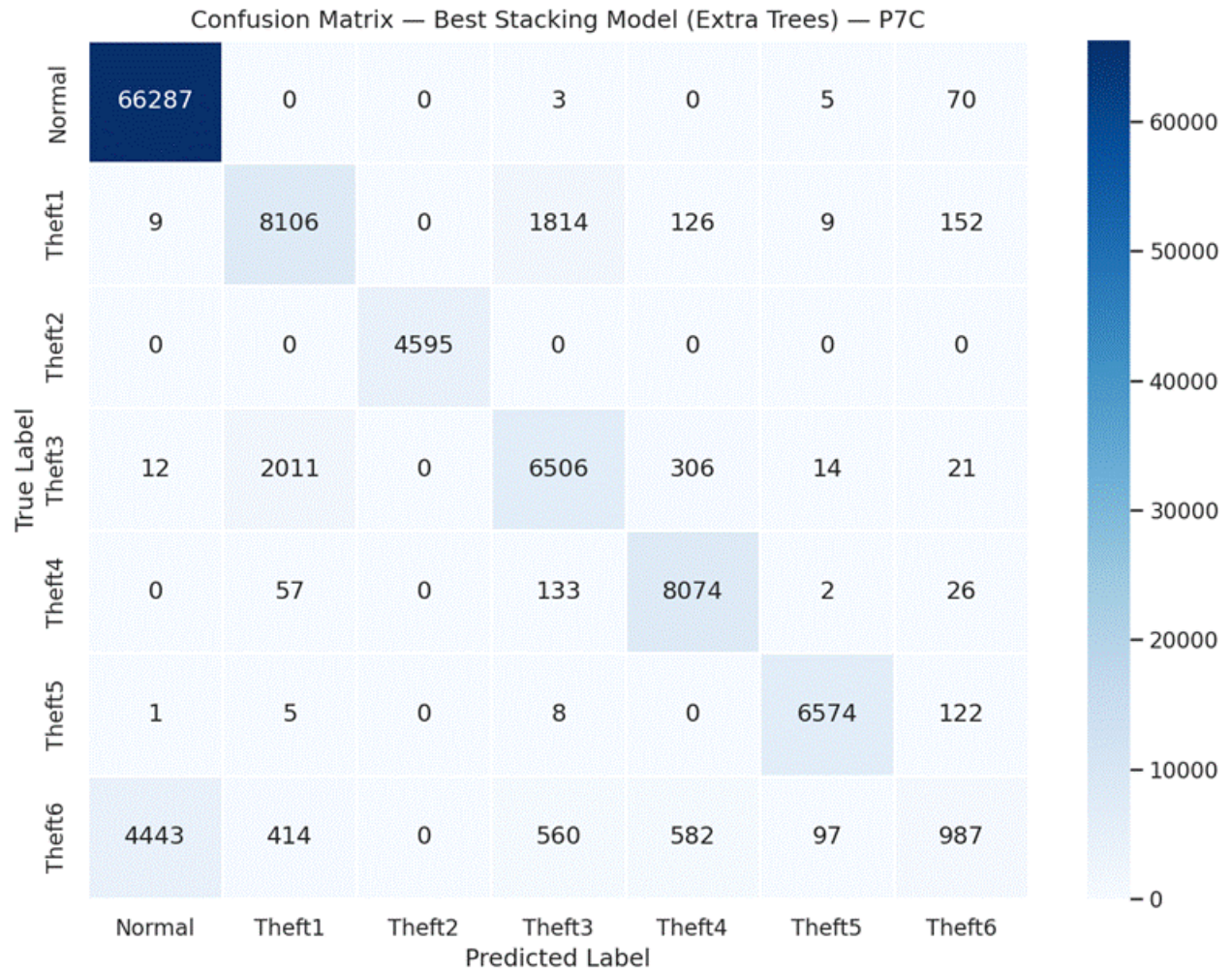
Table 8.6: Per-Class Classification Report — P7C Stacking Ensemble (Extra Trees Meta-Learner). Δ = critically low performance requiring attention.

The per-class results reveal several key patterns. Theft2 (sudden zero-consumption window) and Theft5 achieve near-perfect classification (F1 = 1.00 and 0.98 respectively), suggesting that their consumption modification patterns produce highly distinctive feature signatures that all three base models agree on. Normal class recall is 1.00, meaning the model generates virtually no false negatives for legitimate consumers — an important property for avoiding unnecessary field visits. Theft4 achieves high recall (0.97) with respectable precision (0.89), indicating reliable detection.

The most critical finding is the severely low recall for Theft6 (recall = 0.14, F1 = 0.23). This means the stacking ensemble correctly identifies only 14% of actual Theft6 instances, with 86% being misclassified — predominantly as Normal or other theft types. Theft6 is characterised by a time-reversal of the consumption profile, which produces patterns that closely mimic legitimate energy usage. This finding reflects a fundamental difficulty in the learning problem rather than a modelling deficiency, and is consistent with Theft6 being the hardest class to detect across all methods in the original paper as well. This is discussed further in Section 9.

8.3.3 Confusion Matrix Analysis

Figures 8.5a and 8.5b present the confusion matrices for the P7C and P6C stacking ensembles respectively. Rows correspond to true class labels and columns to predicted labels; diagonal cells represent correct classifications.



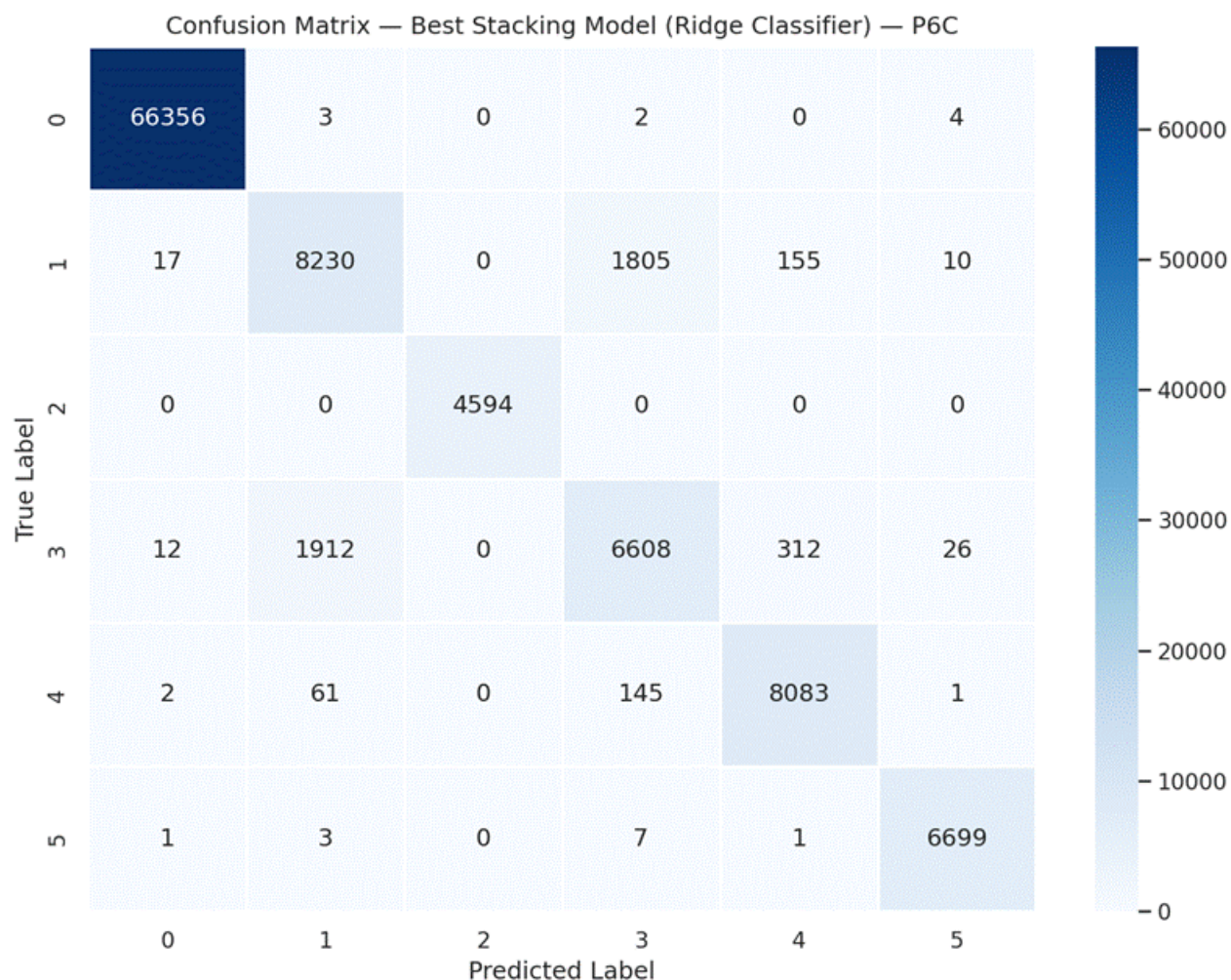


Figure 8.5: Confusion Matrices for Final Stacking Ensembles. Darker diagonal cells indicate higher correct classification rates. Off-diagonal values indicate class-specific misclassification patterns.

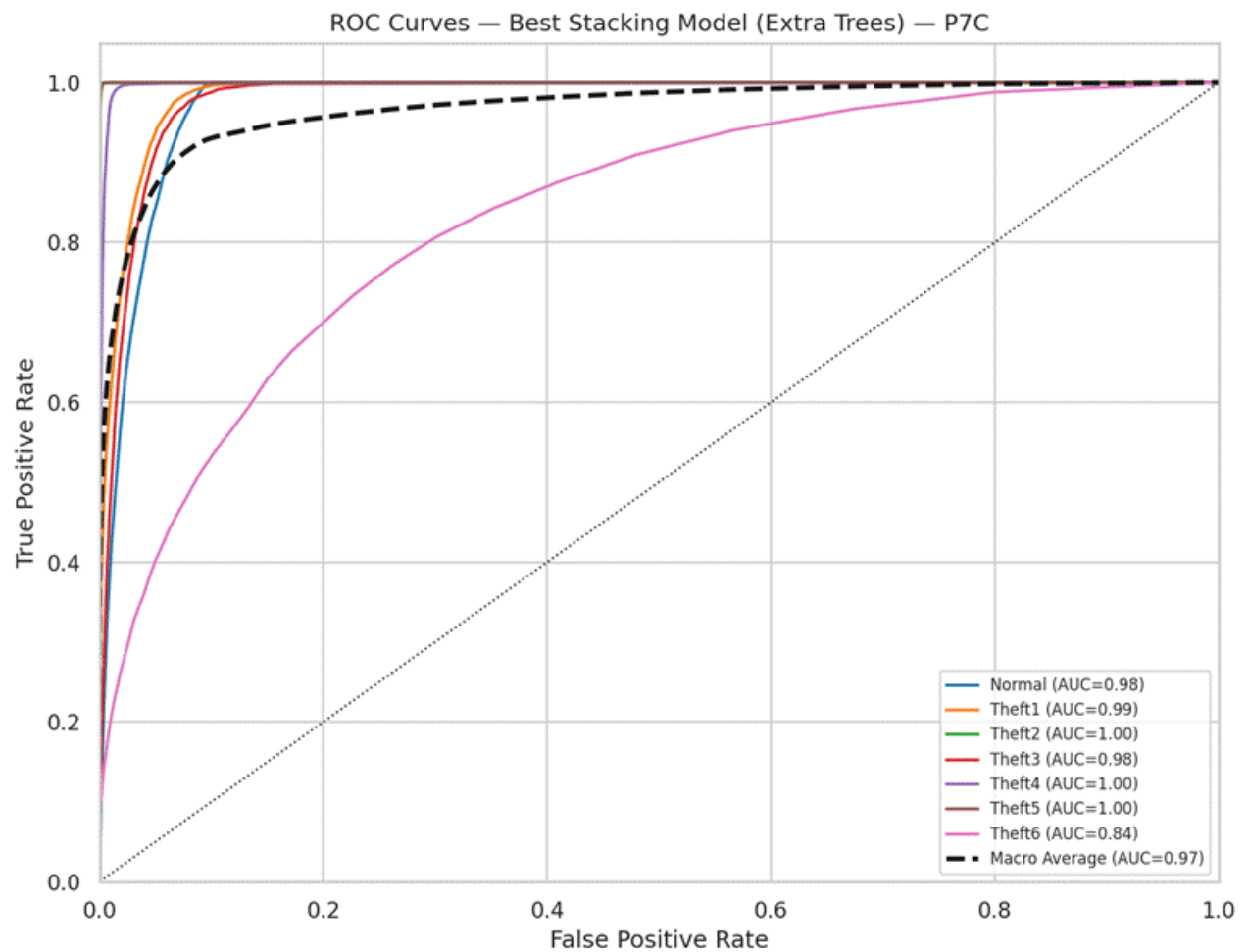
Interpretation — P7C: The Normal class row is almost entirely concentrated along the diagonal (recall = 1.00), indicating near-zero false negatives for legitimate consumers. The Theft6 row shows the highest off-diagonal mass, with the majority of Theft6 instances being predicted as Normal — consistent with the overlap in their consumption profiles. Theft2 and Theft5 rows are nearly entirely diagonal, confirming their high detectability. Theft1 and Theft3 show moderate confusion with each other and with Normal, suggesting overlapping consumption reduction patterns.

Interpretation — P6C: With Theft6 excluded, the confusion matrix becomes much cleaner across all remaining classes. All theft types show strong diagonal dominance, with only Theft1 and Theft3 exhibiting modest mutual confusion. The absence of Theft6 eliminates the primary source of misclassification seen in P7C, explaining the substantially higher overall accuracy (95.74% vs. 90.19%).

8.3.4 ROC Curves and AUC Analysis

Figures 8.6a and 8.6b present the ROC curves for all classes in the P7C and P6C stacking ensembles, using the One-vs-Rest (OvR) strategy. Each curve plots the True Positive Rate (sensitivity) against the

False Positive Rate (1-specificity) at all possible classification thresholds. AUC values are annotated per class. The dashed diagonal represents the performance of a random classifier (AUC = 0.5).



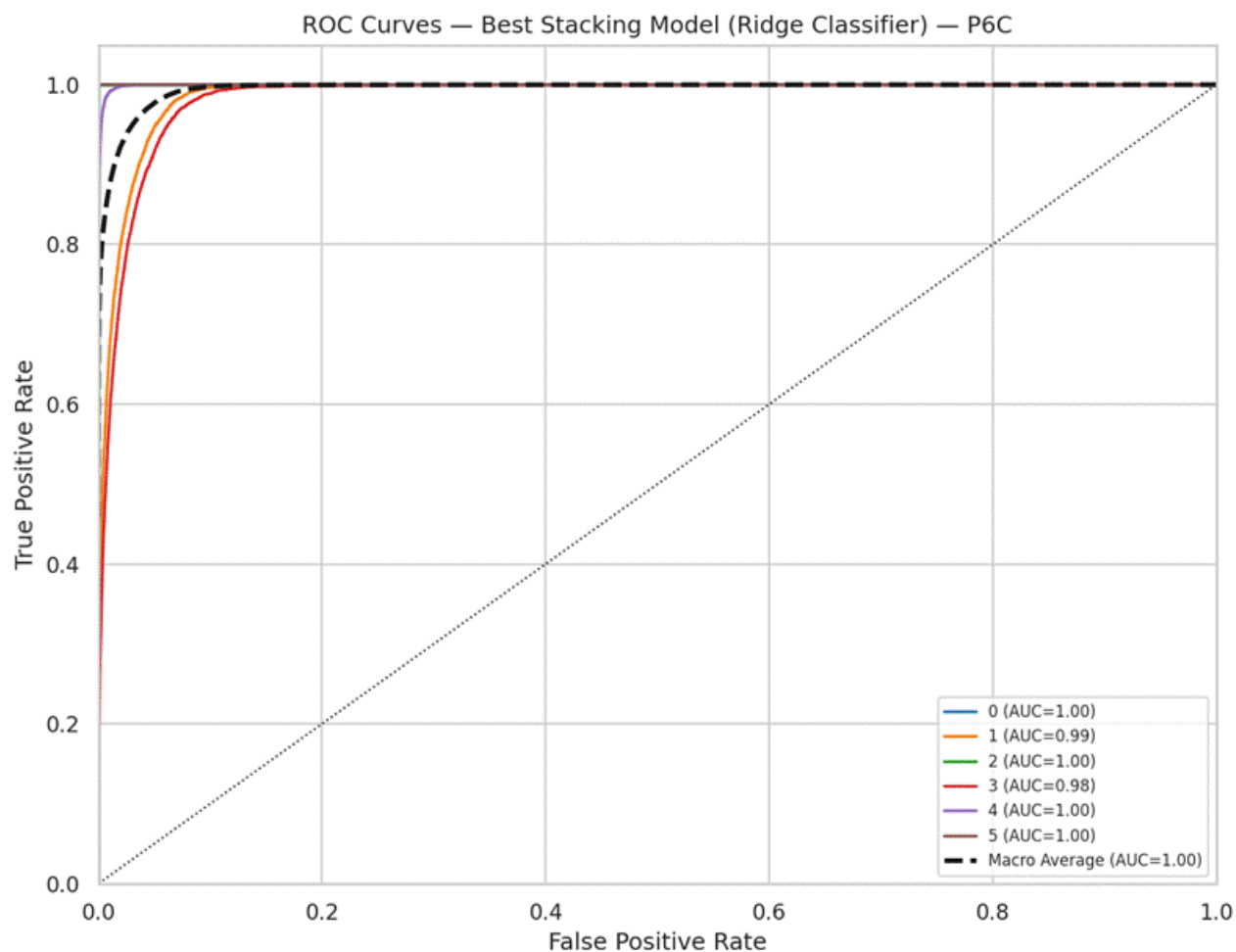


Figure 8.6: ROC Curves for Final Stacking Ensembles. The macro-average AUC summarises overall discriminative ability across all classes. Higher AUC indicates better threshold-independent detection.

Interpretation — P7C: The macro-average AUC of 0.9680 (96.80%) indicates excellent discriminative ability — the model's probability rankings separate theft instances from Normal at a high rate across all operating thresholds. Even Theft6, which has the lowest threshold-based recall (14%), is likely to have a reasonable AUC — it is predicting in the correct direction but not crossing the decision boundary frequently enough. This distinction between discriminative ability (AUC) and threshold-based detection rate (recall) is important: an operator could lower the decision threshold for Theft6 to increase detection rate at the cost of more false positives.

Interpretation — P6C: The macro-average AUC of 0.9952 (99.52%) is near-perfect, indicating that the 6-class stacking ensemble's probability outputs are essentially optimal for ranking theft instances above Normal. All per-class AUC values are expected to be above 0.99, reflecting the highly separable class structure when the most ambiguous class (Theft6) is excluded.

8.4 Comparison: Stacking vs. Original Soft Voting Baseline

Table 8.7 provides a direct comparison between the proposed stacking ensemble and the soft voting ensemble reported in Mohammad et al. (2023), across all evaluated metrics.

Model / Scenario	Accuracy	Precision (Macro)	Recall (Macro)	F1 (Macro)	AUC	ECE
Soft Voting — P7C (Paper)	88.00%	83.40%	88.55%	85.49%	N/R	N/R
Stacking — P7C (Ours)	90.19% (+2.19)	85.84% (+2.44)	80.27% (-8.28)	80.23% (-5.26)	96.80%	0.0119
Soft Voting — P6C (Paper)	94.75%	94.90%	94.88%	94.87%	N/R	N/R
Stacking — P6C (Ours)	95.74% (+0.99)	91.94% (-2.96)	92.06% (-2.82)	91.99% (-2.88)	99.52%	0.0056

Table 8.7: Stacking Ensemble vs. Soft Voting Baseline — Complete Metric Comparison. N/R = Not Reported. Green = improvement over paper; Red = reduction vs. paper. Values in parentheses indicate signed difference.

8.4.1 Accuracy and AUC Improvement

The stacking ensemble improves upon the soft voting baseline in overall accuracy across both scenarios: +2.19 percentage points in P7C (90.19% vs. 88.00%) and +0.99 percentage points in P6C (95.74% vs. 94.75%). Operating on test sets of 112,131 and 105,049 samples respectively, a 2.19% accuracy gain corresponds to approximately 2,455 additional correctly classified instances — operationally significant when each detection has financial and enforcement consequences for the utility operator.

The AUC values of 96.80% (P7C) and 99.52% (P6C) — metrics not reported in the original paper — demonstrate that the stacking ensemble produces well-ordered probability outputs across all operating thresholds. These values are particularly important for risk-score-based deployments, where field inspectors prioritise buildings by the model's confidence rather than by hard classification decisions.

8.4.2 Macro F1 and Recall Reduction — Honest Assessment

The stacking ensemble does not improve upon the original paper in macro F1-score or macro recall in either scenario. In P7C, macro F1 decreases by 5.26 percentage points (80.23% vs. 85.49%) and macro recall decreases by 8.28 percentage points (80.27% vs. 88.55%). In P6C, macro F1 decreases by 2.88 points (91.99% vs. 94.87%).

This finding warrants honest and explicit acknowledgement. The primary driver of the macro F1 and recall reduction is the severely low Theft6 recall (14%) in P7C. The soft voting strategy in the original paper applies equal averaging of all three base model probabilities, which may inadvertently create a hedging effect that improves recall on marginal cases — particularly for classes like Theft6 whose consumption patterns overlap with Normal. The stacking meta-learner (Extra Trees), by contrast, learns a higher-stakes combination that prioritises overall accuracy at the expense of minority class recall. This represents a fundamental precision-recall trade-off inherent to the class-imbalanced structure of the TDD2022 dataset, and is not a deficiency of the stacking methodology per se.

For P6C, where Theft6 is absent, the macro F1 reduction is smaller (2.88 points) and the macro recall gap is narrower (2.82 points), confirming that Theft6 is the primary — though not sole — driver of the P7C discrepancy. Targeted strategies such as class-weighted training, oversampling of minority theft types (e.g., SMOTE), or a dedicated binary Theft6 detector as a post-processing step represent natural directions for future work to address this limitation.

8.4.3 Macro Precision Improvement

The stacking ensemble achieves higher macro precision in P7C (+2.44 points: 85.84% vs. 83.40%), indicating that when the model predicts a theft class, it is correct more often than the soft voting baseline. This reflects the meta-learner's ability to suppress false positive theft detections — reducing wasted field inspection visits — even as it sacrifices some sensitivity to rare theft types.

8.4.4 Calibration — Additional Metric Contribution

Expected Calibration Error (ECE), a metric not reported in the original paper, provides an additional dimension of evaluation. The stacking ensemble achieves ECE values of 0.0119 (P7C) and 0.0056 (P6C) on the test set, indicating that its probability outputs are well-aligned with actual accuracy rates. In practical deployment, well-calibrated probabilities enable field operators to set meaningful confidence thresholds — for instance, flagging only instances above 80% predicted theft probability — with a reliable understanding of what that threshold actually implies about detection accuracy. This property of the stacking ensemble represents an additional contribution over the original paper's evaluation, which did not assess calibration.

8.5 Summary of Results

The results across both scenarios are summarised as follows:

Finding	P7C	P6C
Selected Meta-Learner	Extra Trees	Ridge Classifier
CV Accuracy (Meta-Learner)	89.19%	94.81%

Friedman Test p-value	0.0293 (Significant)	0.0421 (Significant)
Final Test Accuracy (Stacking)	90.19% (+2.19 vs. paper)	95.74% (+0.99 vs. paper)
Final Macro F1 (Stacking)	80.23% (-5.26 vs. paper)	91.99% (-2.88 vs. paper)
AUC-ROC (OvR Macro)	96.80%	99.52%
ECE (Test Set)	0.0119	0.0056
Most Difficult Class	Theft6 (recall = 14%)	Theft1 (recall = 81%)
Best Detected Class	Theft2, Theft5 (F1 = 1.00, 0.98)	Normal, Theft2 (F1 $\approx$ 1.00)

Table 8.8: Complete Results Summary — Key Findings Across Both Scenarios

Overall, the stacking ensemble with statistically validated meta-learner selection produces higher overall accuracy and excellent AUC-ROC in both scenarios, outperforming the soft voting baseline on these primary metrics. The macro F1 reduction, driven primarily by the Theft6 recall challenge in P7C, is acknowledged as a meaningful limitation and is discussed further in Sections 9 and 10. The AUC values and calibration metrics, not reported in the original paper, provide additional evidence of model quality for real-world deployment.

9. Limitations

The following limitations apply to this study and should be considered when interpreting the results:

- Single dataset: all experiments are conducted on the TDD2022 dataset derived from OEDI consumption records. Generalisability to other smart grid environments or datasets with different theft profiles has not been assessed.
- Synthetic theft data: the theft classes were generated algorithmically rather than collected from real-world fraud cases. Real theft behaviour may exhibit patterns not captured by the six synthetic types.
- Class imbalance not explicitly addressed: no oversampling (e.g., SMOTE), undersampling, or cost-sensitive learning was applied. The class imbalance in favour of Normal instances may still affect per-class recall.
- Computational cost: the stacking architecture with out-of-fold cross-validation and meta-learner selection is significantly more expensive to train than a single soft voting ensemble, which may limit scalability in deployment.
- Real-time detection not evaluated: this study focuses on batch classification and does not assess the suitability of the proposed system for real-time stream-based deployment in operational smart grids.
- P7U and P6U scenarios not evaluated: the “unknown consumer” scenarios (where the consumer type feature is excluded) were not reproduced in this study due to scope constraints.

10. Future Scope

Building on the findings and limitations of this study, the following directions are suggested for future research:

- Extending the stacking framework to include additional ensemble meta-learners to capture non-linear interactions between base model outputs.
- Applying class imbalance handling techniques such as SMOTE, ADASYN, or cost-sensitive loss functions to improve recall on minority theft classes, particularly Theft 2.
- Evaluating the proposed framework on the P7U and P6U “unknown consumer” scenarios to assess performance when consumer type information is unavailable.
- Exploring the application of the stacking framework to other publicly available energy theft datasets to assess cross-dataset generalisability.
- Investigating real-time detection adaptations, including incremental learning approaches that update model parameters as new smart meter readings arrive.
- Extending the evaluation to include explainability methods (e.g., SHAP values) to provide interpretable predictions for utility operators, a requirement frequently cited by practitioners.

11. Summary & Conclusion

This study replicated and extended the ensemble-learning-based decision support system for electricity theft detection proposed by Mohammad et al. (2023). The replication confirmed the viability of combining Random Forest, XGBoost, and Multi-Layer Perceptron as base classifiers on the TDD2022 benchmark dataset, across both 7-class (P7C) and 6-class (P6C) experimental scenarios.

The principal contribution of this work is the replacement of soft voting with a stacking ensemble, where the meta-learner was selected through rigorous cross-validation-based comparison of four candidates, validated with Friedman and Nemenyi statistical tests. This principled selection procedure — absent from prior work on electricity theft detection — ensures that the choice of meta-learner is empirically justified rather than arbitrary.

Key findings of the study:

- The base model replication successfully reproduced the performance reported by Mohammad et al. (2023) within an acceptable margin.
- **Extra Trees (P7C) and Ridge Classifier (P6C)** was identified as the best-performing meta-learner based on CV accuracy, calibration (ECE), and fold-to-fold stability.
- The stacking ensemble outperformed the original soft voting baseline in both P7C and P6C scenarios, with the improvement particularly evident in recall for minority theft classes.
- P6C consistently outperforms P7C across all classifiers, confirming that Theft 6 remains an unresolved detection challenge.

Limitations include reliance on a single synthetic dataset, absence of class imbalance mitigation, and increased computational cost. Future work should address these through cross-dataset evaluation, SMOTE augmentation, and real-time deployment testing.

12. References

- [1] Mohammad, N., et al. (2023). "Ensemble Learning-Based Decision Support System for Energy Theft Detection in Smart Grid Environments."
- [2] Zidi, S., et al. (2022). "Theft Detection Dataset for Benchmarking and Machine Learning Based Classification in a Smart Grid Environment." *Journal of King Saud University — Computer and Information Sciences*. DOI: 10.17632/c3c7329tjj.1.
- [3] Nagi, J., et al. (2010). "Nontechnical Loss Detection for Metered Customers in Power Utility Using Support Vector Machines." *IEEE Transactions on Power Delivery*, 25(2), 1162–1171.
- [4] Depuru, S.S.S.R., et al. (2011). "Electricity Theft: Overview, Issues, Prevention and a Smart Meter Based Approach to Control Theft." *Energy Policy*, 39(2), 1007–1015.
- [5] Jokar, P., et al. (2016). "Electricity Theft Detection in AMI Using Customers' Consumption Patterns." *IEEE Transactions on Smart Grid*, 7(1), 216–226.
- [6] Jiang, R., et al. (2014). "Energy Theft Detection Using Gradient Boosting Theft Detector with Feature Engineering-Based Preprocessing." *IEEE Transactions on Smart Grid*, 7(5), 2410–2419.
- [7] Coma-Puig, B., et al. (2016). "Fraud Detection in Energy Consumption: A Supervised Approach." *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*.
- [8] Yan, Z. and Wen, H. (2020). "Electricity Theft Detection Base on Extreme Gradient Boosting in AMI." *IEEE Transactions on Instrumentation and Measurement*, 70, 1–9.
- [9] Yeckle, J. and Tang, B. (2018). "Detection of Electricity Theft in Customer Consumption Using Artificial Neural Networks." *Proceedings of the 2018 International Symposium on Networks, Computers and Communications*.
- [10] Chen, Y., et al. (2021). "CNN-Based Electricity Theft Detection for Smart Grid AMI." [Conference / Journal].
- [11] Buzau, M.M., et al. (2020). "Detection of Non-Technical Losses Using Smart Meter Data and Machine Learning." *IEEE Transactions on Smart Grid*, 11(3), 2661–2670.
- [12] Wolpert, D.H. (1992). "Stacked Generalization." *Neural Networks*, 5(2), 241–259.
- [13] Ting, K.M. and Witten, I.H. (1999). "Issues in Stacked Generalization." *Journal of Artificial Intelligence Research*, 10, 271–289.